

推动计划

采用下面的代码复用路线：

代码块

- 1 SemLayoutDiff 负责 3D-FRONT / architecture-conditioned semantic layout 预处理与 baseline
- 2 EditRoom 负责编辑扰动、editing pair 和语言引导编辑 baseline
- 3 Qwen-VL 官方 qwen-vl-finetune 负责 VLM Constructor / Reviewer / Planner 训练
- 4 DiffSynth-Studio/Qwen-Image-Blockwise-ControlNet-Inpaint 负责局部修复执行器训练

最重要的变化是：

代码块

- 1 原来：
- 2 InstantX/Qwen-Image-ControlNet-Inpainting + 自己改 Diffusers 训练脚本（因为没有）
- 3
- 4 现在：
- 5 DiffSynth-Studio/Qwen-Image-Blockwise-ControlNet-Inpaint + 现成 LoRA/full training 脚本

当前主训练路线应该是 DiffSynth-Studio，而不是先去改 Diffusers 的 train_controlnet.py

1. 最终模块划分

1.1 总表

模块	采用代码	作用	是否主线	是否需要自己写训练框架
VLM Constructor / Reviewer / Planner	Qwen-VL 官方 qwen-vl-finetune	训练 Qwen2.5-VL 多任务 LoRA	是	否
Architecture condition / 3D-FRONT 预处理	SemLayoutDiff	提取建筑条件、语义图、baseline	是	否，主要写转换器
编辑扰动 / editing pair	EditRoom	生成编辑前后 pair、参考编辑协议	是	否，主要改输出格式
初始生成器	你已有 Qwen-Image LoRA	Ours-Initial	是	已有，不重训优先
局部修复执行器	DiffSynth-Studio Qwen-Image-Blockwise-ControlNet-Inpaint	mask-based local repair	是	否，直接改 metadata
InstantX Qwen-Image-ControlNet-Inpainting	InstantX HF 模型	zero-shot executor baseline	否	否
SDXL / FLUX / kohya / Diffusers ControlNet	fallback	非 Qwen 快速备选	否	可选

最终主线：

代码块

```
1  Qwen2.5-VL shared LoRA
2  + 已有 Qwen-Image initial LoRA
3  + DiffSynth Qwen-Image Blockwise ControlNet Inpaint LoRA
4  + SemLayoutDiff / EditRoom 数据与 baseline
5  + LoReflection 自己的 Observer / LoReview / LoRAM / RepairPlan / Mask Adapter / Evaluator
```

2. 模块一：VLM 训练代码

2.1 结论

VLM 训练直接用 Qwen-VL 官方 `qwen-vl-finetune`。

要训练的是一个 shared Qwen2.5-VL LoRA，承担三个任务：

代码块

```
1 [TARGET_STATE_CONSTRUCTION]
2 [SOFT_VERIFICATION]
3 [CORRECTION_PLANNING]
```

其中当前最优先训练的是：

代码块

```
1 [CORRECTION_PLANNING]
```

因为它直接决定：

代码块

```
1 LoReview
2 → RepairPlan
3 → mask_spec
4 → correction_prompt
5 → ControlNet repair sample
```

2.2 数据格式

把你的样本转成 Qwen-VL 的 `image + conversations` 格式：

代码块

```
1 {
2   "image": ["images/bad_layout_0001.png"],
3   "conversations": [
4     {
5       "from": "human",
6       "value": "<image>\n[CORRECTION_PLANNING]\nArchitecture summary=
{...}\nGoal LoState={...}\nObserved LoState={...}\nLoReview={...}\nPlease
output RepairPlan JSON with action_type, target_ref, protected_refs,
mask_spec, correction_prompt, and acceptance_criteria."
7     },
8     {
9       "from": "gpt",
10      "value": "{...RepairPlan JSON...}"
11    }
12  ]
13 }
```

```
11     }
12 ]
13 }
```

2.3 第一版训练命令

代码块

```
1 torchrun --nproc_per_node=4 \
2     qwen_vl/train/train_qwen.py \
3     --model_name_or_path Qwen/Qwen2.5-VL-3B-Instruct \
4     --dataset_use lostate_planner_v1%100 \
5     --output_dir checkpoints/qwen25vl_lostate_planner_lora \
6     --bf16 \
7     --per_device_train_batch_size 2 \
8     --gradient_accumulation_steps 8 \
9     --learning_rate 2e-7 \
10    --num_train_epochs 3 \
11    --lora_enable True \
12    --lora_r 8 \
13    --lora_alpha 16 \
14    --lora_dropout 0.0
```

2.4 需要写什么

只写数据转换脚本：

代码块

```
1 tools/export_to_qwen_vl_sft.py
```

输入：

代码块

```
1 planner_sft_v1/*.json
```

输出：

代码块

```
1 qwen_vl_sft_planner_v1.json
```

3. 模块二：SemLayoutDiff

3.1 结论

SemLayoutDiff 不作为 LoReflection 主方法，但必须复用它的 3D-FRONT 预处理、architecture-conditioned semantic layout 数据组织方式和 baseline。

3.2 你要拿什么

SemLayoutDiff 部分	你怎么用
3D-FRONT / 3D-FUTURE preprocessing	生成 object layout、room metadata、category mapping
architecture-conditioned 2D semantic layout	参考 Architecture Condition Image
semantic color index	对齐你的 palette / category registry
train_sldn.py	作为 architecture-conditioned generation baseline
sample_layout_t.py	生成 SemLayoutDiff baseline 输出
APM	如果需要 2D semantic map lifting 到 3D，可参考

3.3 重点代码路径

代码块

```
1 SemLayoutDiff/preprocess/scripts/data_processor.py
2 SemLayoutDiff/preprocess/scripts/data_to_numpy.py
3 SemLayoutDiff/preprocess/scripts/pickle_threed_front_dataset.py
4 SemLayoutDiff/preprocess/scripts/json_threed_future_dataset.py
5 SemLayoutDiff/render_dataset_improved_mat.py
6 SemLayoutDiff/scripts/train_sldn.py
7 SemLayoutDiff/scripts/sample_layout.py
```

3.4 使用方式

不要把 SemLayoutDiff 改成 LoReflection 主仓库核心。正确方式是：

代码块

```
1  third_party/SemLayoutDiff
2  → 提取 architecture / semantic layout / baseline output
3  → 转成 LoReflection schema
```

你要写转换器：

代码块

```
1  tools/convert_semlayoutdiff_to_loreflection.py
2  tools/build_arch_json_from_semlayoutdiff.py
3  tools/build_semantic_layout_from_semlayoutdiff.py
```

3.5 直接可跑命令

训练 SemLayoutDiff baseline：

代码块

```
1  python scripts/train_sldn.py \
2  --config configs/sldn/mixed_con_unified_config.yaml
```

采样 layout baseline：

代码块

```
1  python scripts/sample_layout.py \
2  --config configs/sldn/sample_layout.yaml
```

4. 模块三：EditRoom

4.1 结论

EditRoom 不作为 LoReflection 主方法，但非常适合用来：

代码块

```
1  1. 生成 editing pair;
2  2. 参考 add / remove / move / replace 等编辑扰动;
3  3. 生成 Planner SFT 数据;
```

不要把 EditRoom 的 diffusion 模型塞进 LoReflection 主方法，否则贡献边界会混乱。

4.2 要拿什么

EditRoom 部分	你怎么用
editing pair generator	生成 I_bad / I_gt pair
perturbation rules	对齐 LoReview issue 和 RepairPlan action
scene graph / scene layout 表示	参考 LoState / LoReview 构造
edit prompt 模板	参考 correction_prompt
editing baseline	作为语言引导编辑对照

4.3 重点代码路径

代码块

```
1 EditRoom/tools/generate_perturbations.py
2 EditRoom/tools/editroomperturb.py
3 EditRoom/tools/generate_prompt.py
4 EditRoom/src/train_edit.py
5 EditRoom/configs/bedroom_sg2sc_diffusion.yaml
6 EditRoom/configs/bedroom_sg_diffusion.yaml
```

4.4 训练命令

代码块

```
1 python3 src/train_edit.py \
2   --config_file configs/bedroom_sg2sc_diffusion.yaml \
3   --output_directory ./weights \
4   --with_wandb_logger
```

然后：

代码块

```
1 python3 src/train_edit.py \
2   --config_file configs/bedroom_sg_diffusion.yaml \
```

```
3 --output_directory ./weights \
4 --with_wandb_logger
```

4.5 你需要自己写什么

代码块

```
1 tools/convert_editroom_pairs_to_loreflection.py
2 tools/build_repairplan_from_editroom_pair.py
3 tools/build_mask_spec_from_editroom_pair.py
```

目标是把 EditRoom 的编辑对转成：

代码块

```
1 I_bad
2 I_gt
3 LoReview
4 RepairPlan
5 mask_spec
6 control_mask
7 correction_prompt
```

5. 模块四：局部修复执行器最终选择

最终采用：

代码块

```
1 主训练 executor:
2 DiffSynth-Studio/Qwen-Image-Blockwise-ControlNet-Inpaint
```

5.2 为什么主路线用 DiffSynth-Studio

DiffSynth-Studio 不是单独模型，而是一个扩散模型训练 / 推理框架。它对 Qwen-Image 系列提供了：

代码块

```
1 inference
2 low-VRAM inference
```


- 3 full training
- 4 full training validation
- 5 LoRA training
- 6 LoRA validation

其中最关键的是它已经有：

代码块

```
1 examples/qwen_image/model_training/train.py
2 examples/qwen_image/model_training/lora/Qwen-Image-Blockwise-ControlNet-
  Inpaint.sh
3 examples/qwen_image/model_training/full/Qwen-Image-Blockwise-ControlNet-
  Inpaint.sh
4 examples/qwen_image/model_inference/Qwen-Image-Blockwise-ControlNet-Inpaint.py
```

这意味着你不需要自己改 Diffusers 的 `train_controlnet.py` 作为第一路线。

5.3 DiffSynth 和 InstantX 的区别

对比项	InstantX/Qwen-Image-ControlNet-Inpainting	DiffSynth/Qwen-Image-Blockwise-ControlNet-Inpaint
框架	Hugging Face Diffusers	DiffSynth-Studio
主要用途	推理 / zero-shot baseline	训练 / LoRA / full fine-tune
模型接口	control_image + control_mask + prompt	input_image + inpaint_mask + blockwise_controlnet_inputs + prompt
训练脚本	模型卡不提供完整训练脚本	有 LoRA 和 full training 脚本
数据格式	你需要自己定义	官方 metadata.csv
当前推荐	对照	主线

5.4 论文里怎么写

不要写死某个仓库名，建议写成：

代码块

```
1 Qwen-Image-based Local Inpainting Executor
```

具体实现写：

代码块

1 We instantiate the local repair executor with a Qwen-Image-based blockwise ControlNet inpainting model and adapt it to fixed-palette semantic layout repair using LoRA.

如果需要对照：

代码块

1 We additionally evaluate InstantX/Qwen-Image-ControlNet-Inpainting as a zero-shot executor baseline.

6. DiffSynth-Studio 局部修复训练

6.1 DiffSynth 数据字段

DiffSynth 的 Qwen-Image Blockwise ControlNet Inpaint LoRA 训练使用：

代码块

1 metadata.csv
2 image
3 blockwise_controlnet_image
4 blockwise_controlnet_inpaint_mask
5 prompt

映射到 LoReflection：

LoReflection 字段	DiffSynth 字段	含义
target_image	image	修复后的 GT semantic layout
control_image / I_bad	blockwise_controlnet_image	有问题的当前 semantic layout
control_mask	blockwise_controlnet_inpaint_mask	白色为要修复区域
correction_prompt	prompt	final-state descriptive prompt

6.2 metadata.csv 示例

代码块

```
1 image,blockwise_controlnet_image,blockwise_controlnet_inpaint_mask,prompt
2 targets/000001_target.png,images/000001_bad.png,masks/000001_mask.png,"A top-
down fixed-palette semantic bedroom layout where the masked region contains
one nightstand beside the bed. All furniture outside the mask remains
unchanged."
3 targets/000002_target.png,images/000002_bad.png,masks/000002_mask.png,"A top-
down fixed-palette semantic bedroom layout where the masked object is replaced
by one desk near the window. All non-target furniture remains unchanged."
```

6.3 你要写的导出脚本

代码块

```
1 tools/export_loreflection_to_diffsynth_inpaint.py
```

输入：

代码块

```
1 data/controlnet_repair_v1/train.json
```

输出：

代码块

```
1 data/diffsynth_loreflection_inpaint_v1/
2 metadata.csv
3 images/
4 masks/
5 targets/
```

核心映射：

代码块

```
1 row["image"] = sample["target_image"]
2 row["blockwise_controlnet_image"] = sample["control_image"]
3 row["blockwise_controlnet_inpaint_mask"] = sample["control_mask"]
```

```
4 row["prompt"] = sample["correction_prompt"]
```

6.4 推荐训练顺序

阶段	数据	目标
D0	官方 example	确认 DiffSynth 环境和脚本能跑
D1	50 repair pairs	overfit, 检查数据格式
D2	1k repair pairs	初步 domain LoRA
D3	5k repair pairs	主实验训练
D4	Planner-mask samples	适应真实 Planner mask
D5	accepted loop logs	闭环继续训练

6.5 第一版 LoRA 命令

先复制 DiffSynth 官方 LoRA 脚本，只改数据路径和输出路径：

代码块

```
1 accelerate launch examples/qwen_image/model_training/train.py \
2   --dataset_base_path data/diffsynth_loreflection_inpaint_v1 \
3   --dataset_metadata_path data/diffsynth_loreflection_inpaint_v1/metadata.csv \
4   --data_file_keys
   "image,blockwise_controlnet_image,blockwise_controlnet_inpaint_mask" \
5   --max_pixels 1048576 \
6   --dataset_repeat 50 \
7   --model_id_with_origin_paths "Qwen/Qwen-
   Image:transformer/diffusion_pytorch_model*.safetensors,Qwen/Qwen-
   Image:text_encoder/model*.safetensors,Qwen/Qwen-
   Image:vae/diffusion_pytorch_model.safetensors,DiffSynth-Studio/Qwen-Image-
   Blockwise-ControlNet-Inpaint:model.safetensors" \
8   --learning_rate 1e-4 \
9   --num_epochs 5 \
10  --remove_prefix_in_ckpt "pipe.dit." \
11  --output_path "./models/train/loreflection_qwen_inpaint_lora_v1" \
12  --lora_base_model "dit" \
13  --lora_target_modules
   "to_q,to_k,to_v,add_q_proj,add_k_proj,add_v_proj,to_out.0,to_add_out,img_mlp.ne
   t.2,img_mod.1,txt_mlp.net.2,txt_mod.1" \
14  --lora_rank 32 \
```

```
15     --extra_inputs  
    "blockwise_controlnet_image,blockwise_controlnet_inpaint_mask" \  
16     --use_gradient_checkpointing \  
17     --find_unused_parameters
```

6.6 训练验证

不要只看 loss，必须看：

代码块

```
1  masked target success  
2  non-target preservation  
3  fixed-palette consistency  
4  Observer parse success  
5  collision change  
6  OOB change  
7  door/window blocking change  
8  Practical Usability SR
```

9. Baseline 代码最终清单

9.1 生成任务 baseline

Baseline	代码用途
SemLayoutDiff	architecture-conditioned semantic layout baseline
ATISS	autoregressive 3D indoor scene synthesis baseline
DiffuScene	diffusion-based text / unconditional / rearrangement baseline
MiDiffusion	floor-plan conditioned layout baseline
LayoutGPT	open-loop LLM layout planning baseline

9.2 编辑任务 baseline

Baseline	代码用途
EditRoom	language-guided editing baseline
DirectLayout Re-generation	global regeneration baseline
InstantX zero-shot executor	zero-shot local repair baseline
DiffSynth LoRA executor	main local repair executor

9.3 不要一次性全做

优先顺序：

代码块

1

第一批：

2

SemLayoutDiff

3

EditRoom

```
4 Ours-Initial
5 Ours-Final
6 w/o Reflection Loop
7 w/o LoRAM
8 w/o AcceptanceController
9
10 第二批:
11 ATISS
12 DiffuScene
13 LayoutGPT
14
15 第三批:
16 MiDiffusion
17 DirectLayout
18 更多 external editing baselines
```

10. 项目代码结构

最终推荐结构:

代码块

```
1 LoReflection/
2   configs/
3     category_registry_v1.yaml
4     palette_v1.json
5     issue_action_mask_map_v1.yaml
6     prompt_templates_v1.yaml
7
8   schemas/
9     architecture_v1.schema.json
10    goal_lostate_v1.schema.json
11    observed_lostate_v1.schema.json
12    prompt_package_v1.schema.json
13    loreview_v1.schema.json
14    repairplan_v1.schema.json
15    mask_spec_v1.schema.json
16    controlnet_sample_v1.schema.json
17    eval_representation_v1.schema.json
18
19  third_party/
20    SemLayoutDiff/
21    EditRoom/
22    DiffSynth-Studio/
```

```
23     qwen-vl-finetune/
24
25     tools/
26         convert_semlayoutdiff_to_loreflection.py
27         convert_editroom_to_loreflection.py
28         export_to_qwenvl_sft.py
29         export_loreflection_to_diffsynth_inpaint.py
30         validate_all.py
31         run_leakage_audit.py
32
33     data_pipeline/
34         build_arch_json.py
35         render_arch_condition.py
36         render_gt_semantic_layout.py
37         build_goal_lostate.py
38         build_prompt_package.py
39         generate_perturbations.py
40         build_observed_lostate.py
41         build_loreview.py
42         build_planner_sft_data.py
43         build_controlnet_repair_pairs.py
44
45     runtime/
46         observer.py
47         prompt_builder.py
48         loram.py
49         track_a_geometry_review.py
50         track_b_semantic_review.py
51         mask_tensor_adapter.py
52         acceptance_controller.py
53         run_closed_loop.py
54
55     vlm/
56         train_qwenvl_lora.sh
57         infer_constructor.py
58         infer_reviewer.py
59         infer_planner.py
60
61     diffusion/
62         train_diffsynth_qwen_inpaint_lora.sh
63         infer_diffsynth_qwen_inpaint.py
64         infer_instantx_qwen_inpaint.py
65
66     eval/
67         convert_to_eval_representation.py
68         eval_semantic.py
69         eval_geometry.py
```



```
70     eval_usability.py
71     eval_editing.py
72     eval_reflection.py
73     summarize_tables.py
74
75     experiments/
76         val50/
77         prompt_pilot/
78         planner_sft_1k/
79         inpaint_lora_50_overfit/
80         inpaint_lora_1k/
81         closed_loop_1k/
```

11. 最小可跑实验顺序

Step 1: 接 SemLayoutDiff

目标：

代码块

- 1 从 3D-FRONT 得到：
- 2 Architecture JSON
- 3 Architecture condition image
- 4 GT semantic layout
- 5 object list

产物：

代码块

- 1 data/arch_json_v1
- 2 data/arch_condition_v1
- 3 data/gt_semantic_layout_v1

Step 2: 接 EditRoom

目标：

代码块

```
1 从 GT layout 得到:
2 I_bad
3 I_gt
4 edit_type
5 natural language edit description
```

再转成:

代码块

```
1 LoReview
2 RepairPlan
3 mask_spec
4 control_mask
5 correction_prompt
```

产物:

代码块

```
1 data/repair_pairs_v1
2 data/planner_sft_v1
```

Step 3: 跑 VLM Planner SFT

目标:

代码块

```
1 bad_layout image + Goal + Observed + LoReview
2 → RepairPlan JSON
```

产物:

代码块

```
1 checkpoints/qwen25vl_planner_lora_v1
```

Step 4: 跑 DiffSynth 50 pair overfit

目标：

代码块

```
1 I_bad + control_mask + correction_prompt
2 → I_gt
```

产物：

代码块

```
1 models/train/loreflection_qwen_inpaint_lora_val50
```

检查：

代码块

```
1 loss 下降
2 mask 区域修对
3 mask 外不漂移
4 输出可被 Observer 回读
```

Step 5: 跑 1k 闭环

流程：

代码块

```
1 Goal LoState
2 → Prompt Builder
3 → 你已有 Qwen-Image initial LoRA
4 → Observer
5 → LoReview / LoRAM
6 → VLM Planner
7 → mask_spec
8 → Mask Adapter
9 → DiffSynth Inpaint LoRA
10 → Observer
11 → AcceptanceController
```

输出：

12. 最终执行路线

当前主路线

代码块

- 1 数据:
- 2 SemLayoutDiff + EditRoom
- 3
- 4 VLM:
- 5 Qwen-VL qwen-vl-finetune
- 6
- 7 初始生成:
- 8 你已有 Qwen-Image LoRA
- 9
- 10 局部修复:
- 11 DiffSynth-Studio/Qwen-Image-Blockwise-ControlNet-Inpaint LoRA
- 12
- 13 对照:
- 14 InstantX Qwen-Image-ControlNet-Inpainting zero-shot
- 15 SemLayoutDiff
- 16 EditRoom
- 17 LayoutGPT / ATISS / DiffuScene 后续补

当前不建议优先做

代码块

- 1 1. 自己改 Diffusers QwenImageControlNet training;
- 2 2. 一开始 full training blockwise_controlnet;
- 3 3. 一开始训练 Prompt Builder 黑盒模型;
- 4 4. 一开始做三模型独立 VLM;
- 5 5. 一开始堆所有 external baselines。

13. 现在立刻该做什么

第 1 件事：fork 三个仓库

代码块

```
1  mkdir -p third_party
2
3  git clone https://github.com/3dlg-hcvc/SemLayoutDiff.git
   third_party/SemLayoutDiff
4  git clone https://github.com/UCSB-AI/EditRoom.git third_party/EditRoom
5  git clone https://github.com/modelscope/DiffSynth-Studio.git
   third_party/DiffSynth-Studio
```

Qwen-VL 训练框架放到：

代码块

```
1  third_party/qwen-vl-finetune
```

第 2 件事：先跑 DiffSynth 官方 example

代码块

```
1  cd third_party/DiffSynth-Studio
2  pip install -e .
3
4  python examples/qwen_image/model_inference/Qwen-Image-Blockwise-ControlNet-
   Inpaint.py
5  bash examples/qwen_image/model_training/lora/Qwen-Image-Blockwise-ControlNet-
   Inpaint.sh
```

目的不是出结果，而是确认：

代码块

```
1  环境能跑
2  模型能加载
3  LoRA 训练脚本能启动
4  metadata.csv 格式理解正确
```

第 3 件事：写 DiffSynth 导出脚本

代码块

```
1 tools/export_loreflection_to_diffsynth_inpaint.py
```

把你的 repair pair 转成：

代码块

```
1 metadata.csv
2 image = target_image
3 blockwise_controlnet_image = control_image
4 blockwise_controlnet_inpaint_mask = control_mask
5 prompt = correction_prompt
```

第 4 件事：做 50 pair overfit

代码块

```
1 50 条 repair pair
2 → DiffSynth metadata.csv
3 → Qwen-Image-Blockwise-ControlNet-Inpaint LoRA
4 → 检查修复质量
```

这是局部修复训练能否成立的最小证明。

第 5 件事：再扩到 1k

如果 50 pair 能 overfit，再生成：

代码块

```
1 1k Planner SFT
2 1k DiffSynth repair pairs
```

然后开始闭环小实验。

14. 最终判断

当前最干净、最快、最符合论文需求的实验路线是：

代码块

- 1 SemLayoutDiff 解决 architecture-conditioned layout 数据和 baseline;
- 2 EditRoom 解决编辑扰动和 editing pair;
- 3 Qwen-VL 官方训练代码解决 VLM Planner;
- 4 DiffSynth-Studio Qwen-Image-Blockwise-ControlNet-Inpaint 解决局部修复训练;
- 5 InstantX 只做 zero-shot executor 对照。

你现在最不该做的是：

代码块

- 1 自己从零写 Qwen-Image-ControlNet-Inpainting 训练脚本。

因为 DiffSynth-Studio 已经给了你 Qwen-Image Blockwise ControlNet Inpaint 的 LoRA 和 full training 入口。你只需要把 LoReflection 的 repair pairs 转成它的 `metadata.csv` 格式，然后先跑 50 pair overfit。